

3rd place solution

Rank	3
Team	W & A
Author	Arunodhayan
Topic ID	679236
Version	Original

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679236>

Retrieved with Kaggle CLI on 2026-07-07.

First and foremost, we would like to express our gratitude to the organizers for hosting this fantastic competition! We are honored to share our solution here. Our approach is primarily based on a highly customized **nnU-Net v2** framework. The core highlights include: an asymmetric patch size strategy for training and inference, two-stage cascade prediction (Cascade 3D), customized Test Time Augmentation (TTA), and a post-processing pipeline based on 3D Hessian matrix features for Ridge Detection.

1. Overall Architecture Pipeline

We built an efficient and robust two-stage cascade 3D image segmentation pipeline:

- **Stage 1 (3D Fullres):** Uses a full-resolution 3D model for initial prediction and performs a weighted ensemble of probability maps from multiple models.
- **Stage 2 (3D Cascade Fullres):** Takes the predictions from Stage 1 as spatial prior information (Previous Stage Predictions) and feeds them into the cascade model for refined prediction.
- **Post-processing:** Combines Distance Transform and 3D Hessian matrix to perform fine-grained topological optimization and subtle structure extraction on the model's output probability maps.

2. Dataset Strategy & Processing

- **Full Dataset Training:** To maximize the model's ability to fit the true data distribution, we abandoned the traditional K-Fold cross-validation (i.e., holding out a validation set) approach. The final submitted models were trained directly on the **entire dataset (Train on all data)** to achieve more robust generalization performance.
- **Pseudo-labeling Experiments:** During the competition, we attempted to use external/unlabeled data to generate pseudo-labels to expand the training set. However, experimental results showed that this strategy did not bring significant performance improvements. To keep the solution concise and efficient, we did not adopt the pseudo-labeling strategy in our final version.
- **Data Augmentation:** Building upon nnU-Net's default augmentation strategies, and tailored to the topological characteristics of the target, we forcefully enabled flipping across all axes (Mirror Axes: 0, 1, 2) and set a relatively high probability for rotation augmentation (Rotation Probability: 0.4-0.8).

3. Model Architecture & Training Strategy

We deeply customized the default nnU-Net Trainer. The core improvements are as follows:

- **Asymmetric Patch Size Strategy:** We utilized the **M** size network architecture. Notably, we introduced a "size decoupling" strategy that was proven highly effective in the CZII competition: **the Patch Size during training was set to 128, while the Patch Size during inference was increased to 192**. This allows the model to maintain a higher Batch Size and iteration efficiency during training, while obtaining a larger receptive field during inference, effectively improving the global consistency of the segmentation results.
- **Topology-Aware Loss Function:** In tasks that heavily focus on structural connectivity, the classic Dice + CE combination often struggles to perfectly preserve complex elongated or mesh-like structures. Therefore, we introduced the **clDice Loss** on top of the default loss. clDice is specifically optimized at the skeleton level for the topological connectivity of tubular/mesh-like structures.

- **Optimizer Configuration:** We abandoned traditional learning rate schedulers and switched to the **RAdamScheduleFree** optimizer. Practice has proven that this optimizer not only makes model convergence smoother but also significantly improves training efficiency.
- **Cascade Model Training Path:** To build the Stage 2 Cascade model, we strictly followed a "coarse-to-fine" training paradigm: first training the Lowres (low-resolution) model, and subsequently training the Cascade model based on it. This ensures that the network correctly learns and utilizes the coarse predictions from the previous stage as reliable spatial priors.

4. Inference & Model Ensemble (TTA)

During the inference stage, to balance prediction accuracy and inference speed, we rewrote the prediction logic in `predict_from_raw_data.py`:

- **Asymmetric Inference and Stage Fusion:** As mentioned earlier, we used a large Patch Size of 192 for inference. In the two-stage pipeline, we directly used the output of the **Fullres model** as the feature prior input for the **Cascade model** to make the final prediction. Empirical evidence shows that this combination can capture the richest image details.
- **Customized TTA (Test Time Augmentation):** Besides conventional mirror flipping, we introduced **in-plane rotation TTA ($\pm 15^\circ$)** via Affine Grid Sample. Ultimately, the prediction for each patch fuses the outputs of the regular, flipped, and $\pm 15^\circ$ rotated versions. **Regarding the choice of rotation angle:** Since a random rotation augmentation of $\pm 30^\circ$ was used during training, our experimental evaluations showed that setting the TTA rotation angle to $\pm 15^\circ$ was the most optimal. We also tried introducing larger rotation augmentations during training, but experiments showed that it actually led to performance degradation.
- **Concurrent Extraction and Multi-Model Ensemble:** We wrote dedicated parallel inference scripts to bind multiple processes to specific GPUs, extracting `.npz` probability maps in parallel in a multi-GPU environment. Subsequently, we performed equal-weight or weighted fusion on Checkpoints from different iteration steps or specific settings.

5. Topology-Level Post-processing

This is a crucial part of our solution that achieved significant score improvements. Considering the strong spatial connectivity of the target, we designed the following powerful post-processing pipeline:

1. **Inverse EDT Dilation:** Utilizing the inverse mapping operation based on the Euclidean Distance Transform (EDT) to perform non-linear dilation on the binarized results.
2. **3D Hessian Ridge Detection:** We view the targets as "ridges" in space. After applying Gaussian smoothing to the predicted volume, we calculate the 3D Hessian matrix and use the matrix's eigenvalues to construct a feature space, thereby extracting structures with tubular features and drastically filtering out unstructured background noise.
3. **Topological Pruning and Consistency Pruning:**
 - **Z-axis Consistency Pruning:** If a voxel is positive on the current Z-axis slice but negative on both its upper and lower adjacent slices, it is judged as a false positive and removed. This step played a decisive role in eliminating inter-slice outlier noise.

- We applied 2D morphological closing operations to the slices to smooth the boundaries, and performed anisotropic 3D closing on the overall volume in the XY plane to repair fractured parts.
- Finally, we removed 3D discrete noise with excessively small volumes.

Special Thanks: It is worth mentioning that many inspirations and core code logic in our post-processing pipeline were referenced and benefited from the [wonderful discussions and code](#) shared by the organizers in the discussion forum. We would like to express our sincere gratitude to the organizers and the community for their selfless sharing!

6. Acknowledgments

Thanks again to the organizers for providing this challenging platform, and also thanks to my teammate [@arunodhayan](#) for their hard work during the model training and exploration process!

Our Best Private LB Submission: It is worth noting that our highest-scoring submission on the Private Leaderboard was achieved using a heavily trained Cascade model configuration. Specifically, this best-performing model was built by first training the `Lowres` network for 8000 epochs, and subsequently training the `cascade` network on top of it for an additional 8000 epochs. <https://www.kaggle.com/code/arunodhayan/nnunet-1-rot-tta-ensemble-cascade-no-fillin-0522f9?scriptVersionId=299432647>

Supplemental Q&A

ArjunB · 2026-02-28T03:07:20.087000

Hi could you share how much of an improvement your postprocessing did to the public and private score?

Arunodhayan · 2026-02-28T03:24:37.873000

Hi could you share how much of an improvement your postprocessing did to the public and private score? I believe around 0.02 to 0.03

Sean Johnson_SP · 2026-02-28T03:48:08.167000

Looking forward to the full writeup! Out of curiosity, is the ridge detection and edt dilation from the script in the Vesuvius python library? It sounds exactly like a label "cleanup" filter i run occasionally

Tom · 2026-02-28T04:30:49.560000

I'm curious about the runtime of your post processing. Hessian-based approach is one of the post proc I've tried, which takes very long.

Maapu · 2026-02-28T06:31:34.023000

i'm curious on this, what was the training speed per batch iteration (256 patch? / 8x A100 run ?) to have the belief to run for 8000 epochs / like you looked on training loss alone for progression or topo score improvement every 10/20 validation when it comes to such scale? In other words like the research mentality / thought process behind the solution.

wpl · 2026-02-28T07:10:04

Our final post-processing code mostly comes from [here](#), and we made some simple modifications based on it. As for the processing time, I estimate it to be around an hour.

wpl · 2026-02-28T07:11:06.707000

Yes, our main post-processing comes from the methods you [shared](#)

Arunodhayan · 2026-02-28T10:13:21.947000

[@rajeshthevar](#) we used 2xH100 : to train 3d low res it took 14hrs and for 3d cascade it took 24hrs to completely train 8000 epochs

CPMP · 2026-02-28T12:53:16.987000

Great result.

What implementation did you use fo Hessian based ridge detection? I spent a bit of time doing like you, extracting eigenvalues from it, and train models on it, but didn't manage to make it work. I wonder what I did wrong.

Nevermind, I see you used https://github.com/ScrollPrize/villa/blob/main/vesuvius/src/vesuvius/image_proc/run/edt_frangi_label.py

It was on my todo list to try it, should have prioritized it.

Arunodhayan · 2026-03-02T09:42:00.813000

[@cpmpml](#) Thankyou. Please find attached the notebook containing our implementation below. <https://www.kaggle.com/code/arunodhayan/nnunet-1-rot-tta-ensemble-cascade-no-fillin-0522f9?scriptVersionId=299432647>